Client: Kinsmen

INSY7315 — Work Integrated Learning

TASK 1: DESIGN DOCUMENTATION

Revised for the final submission (Appendix 1)

Team name: Emerald Solutions

Project: Kinsmen Barbershop — Website and Booking System

Module: Information Systems 3E (Work Integrated Learning) — INSY7315, BCA3, The Independent Institute of Education (Emeris)

Date: November 2026

Team member	Student number
Zario Di Paolo	ST10441349
Kaehil Indurjeeth	ST10438880
Kyra Naidoo	ST10448414
Gregory Luyckfasseel	ST10441344

Source code: https://github.com/EMKNDN/emkndn-insy7315-2026-insy7315-project-kaehil-i

Live system: https://kinsmen-web.onrender.com

Table of Contents

Revision summary 4

1. Introduction 6

1.1 Background 6

1.2 Problem statement 6

1.3 Aim and objectives 6

1.4 Scope 6

1.5 Stakeholders 7

1.6 Method 7

1.7 Assumptions and constraints 7

1.8 How to read this document 7

2. Requirements 8

2.1 User roles 8

2.2 User stories 8

2.3 User experience journey map 9

2.4 Work breakdown structure for handover 10

2.5 Assumptions register 11

3. Non-functional requirements 13

4. Analysis artefacts 14

4.1 Domain class diagram 14

4.2 Design decisions 15

5. Implementation 16

5.1 Sequence diagrams 16

5.2 State diagram: booking status 19

5.3 Deployment plan 20

6. Data schema 22

6.1 Choice of technology 22

6.2 Collections and indexes 23

6.3 Data integrity 23

7. Architecture artefacts 24

7.1 Design patterns 24

7.2 Architecture patterns 24

7.3 Cloud architecture 25

8. Security 26

8.1 Rating scales 26

8.2 Risk register 26

8.3 Controls in detail 27
8.4 Privacy (POPIA) 28
8.5 Future security options 28
9. DevOps 29
9.1 Branching and review 29
9.2 Continuous integration 29
9.3 Automated testing 29
9.4 Continuous deployment 29
9.5 Configuration and secrets 30
9.6 Monitoring, release and rollback 30
10. Running costs 31
10.1 Exchange rate and sources 31
10.2 Monthly cost scenarios 31
10.3 What the numbers mean for the shop 31
10.4 Costs not included 32
11. Change management 33
11.1 Roadmap 33
11.2 Preparing staff 33
11.3 Preparing customers 34
11.4 Measuring adoption 34
11.5 Adoption risks 34
12. Prototype and live system 35
References 35

Revision summary

This document replaces the Task 1 report that was submitted in August 2026. The original report described a system that had not yet been built, and several of its choices (Azure hosting, Azure SQL, a shopping cart and checkout) changed during Tasks 2 and 3. This revision describes the system as it was designed, built and deployed, and it answers each point in the lecturer's feedback on the original report.

Writing conventions. Decisions and work that are complete are written in the past tense. How the finished system behaves is written in the present tense. Work that has not been done is written with "will" or "should" and is listed in the roadmap (Section 11). Every section opens with a short introduction, and every requirement carries a status: **Delivered**, **Not built (out of scope)** or **Future**.

Lecturer feedback	What changed in this revision	Where
Some sections have an intro, others do not; tenses change	Every section now opens with an introduction, and one tense convention is used throughout.	All sections
Introduction is missing key points	Rewritten with background, problem statement, aim, objectives, scope, stakeholders, method, assumptions and a reading guide.	Section 1
Roles and stories contradict each other ("confirmed" versus "pending"); need more depth	One evidence-based status for every role and story, an assumptions register, and 28 user stories with acceptance criteria, priority and status.	Sections 2.1, 2.2, 2.5
Journey map: is it a map or a table?	A real journey map (stages, actions, touchpoints, feelings, pain points, system response) plus per-role journeys.	Section 2.3
WBS: wrong levels, mixed up with a network diagram; should be for the handover project	A three-level handover WBS with a WBS dictionary.	Section 2.4
NFRs generic; double-booking claim points to a control that does not exist	Measurable NFRs with how each is verified. Double-booking is explained as it is really enforced.	Section 3
Domain class diagram needs more explanation	Updated class diagram with a full explanation of every class, relationship and design decision.	Section 4
Only one sequence flow; need guest, customer, barber, admin	Four sequence diagrams plus a state diagram with transition guards.	Section 5.1–5.2
Deployment plan: no diagrams, no steps, unclear environment	Environment table, two diagrams and an ordered step-by-step plan with rollback.	Section 5.3
Choice of technology: no references, alternatives, or scalability	Options compared against weighted criteria, with references and a scalability path.	Section 6.1
Design patterns: contradiction about WhatsApp; proof-reading	One consistent statement: email is built; WhatsApp is a future option only. Patterns tied to real code.	Section 7.1
Architecture patterns basic; connect the dots	How the patterns work together, traced through one request.	Section 7.2
Cloud architecture: more justification, tense	Justification per choice, alternatives considered, past tense for what was built.	Section 7.3
Security: formal risk analysis	Ten-risk register with likelihood, impact, rating, controls and residual risk, plus a heat map.	Section 8
DevOps: more detail	Branching, rulesets, pipeline stages, environments, secrets, testing, release and rollback.	Section 9

Lecturer feedback	What changed in this revision	Where
Running costs: old exchange rate, more detail, check prices	Current prices and a dated exchange rate, three scenarios, annual totals and sources.	Section 10
Change management: cleaner roadmap, more detail	Roadmap figure, phase table, training, communication and adoption measures.	Section 11

1. Introduction

This section explains what the project is, why it exists and how the rest of the document is organised. It is the section to read first.

1.1 Background

Kinsmen Barbers is a classic barbershop at Shop 5, 8 Mackeurtan Avenue, Durban North. It offers blade and tapered haircuts, beard trims, hot-towel preparation and a straight-razor finish, and it sells grooming products in the shop. Some members of our team are regular customers, which is why the business was chosen. The owner agreed to work with us after an introduction on 12 April 2026, and a partnership agreement was sent to the business on 21 April 2026.

The shop's appointments were arranged by phone and message, and its products were sold only at the counter. The existing arrangements gave the owner no single view of how busy each barber was, which services sold best or which products were running out.

1.2 Problem statement

Manual booking limits the shop in four ways: customers can only book while someone is free to answer, barbers cannot see or control their own schedule, double bookings and missed appointments are not caught until the customer arrives, and the owner has no record from which to plan staffing or stock.

1.3 Aim and objectives

Aim: to design, build and deploy a web-based booking and shop-information system that lets customers book appointments at any hour and gives the shop's staff the tools to run those bookings.

- **O1.** Let customers see services, barbers and products without an account and book with an account.
- **O2.** Prevent two bookings for the same barber and time, and respect each barber's skills, hours and time off.
- **O3.** Give barbers a schedule they can act on and give the owner administration and reporting tools.
- **O4.** Send customers a confirmation and a reminder so that fewer appointments are missed.
- **O5.** Let customers reserve shop products with a booking and keep stock accurate.
- **O6.** Deploy the system to the cloud at a running cost the shop can afford, with an automated build-and-test pipeline.

1.4 Scope

In scope (built)	Out of scope (not built)
Public pages for services, barbers and products Accounts and sign-in through Auth0 Online booking with multiple services and barber choice Reschedule, cancel, review and rebook Barber schedule, time blocks and status updates Admin catalogue, barbers, products, stock, staff roles and dashboard Email confirmations and 24-hour reminders Cloud deployment with automated tests	Online card payment and delivery (products are reserved and paid for in store) Shopping cart and checkout Loyalty points Guest booking without an account WhatsApp messages and biometric login (future options only) Multi-branch operation

1.5 Stakeholders

Stakeholder	Interest	How they were involved
Owner (client)	More bookings, less admin, accurate stock	Introduced on 12 April 2026; questionnaire emailed on 21 July 2026; asked on 6 October 2026 for the product list and quantities, and referred us to the shop for them.
Barbers	A clear schedule and control over their time	Requirements taken from the questionnaire and from the team's visits as customers.
Customers	Fast, reliable booking	Journeys and stories written from customer experience; tested with team accounts.
Team (Emerald Solutions)	A complete, marked project	Four developers sharing design, build, testing and documentation.
Lecturer	A well-evidenced, professional project	Marks the three tasks and the expo.

1.6 Method

The team worked in iterations that followed the three assessment tasks. Task 1 produced the requirements, design and a clickable prototype. Task 2 delivered the working core: booking, accounts, administration and cloud deployment. Task 3 added the shop, customer profile, dashboard and email reminders, and hardened the deployment. Work was shared through GitHub: each person built a feature on a branch, opened a pull request, had it reviewed by a teammate and merged it only when the automated checks passed (Section 9).

1.7 Assumptions and constraints

- Kinsmen is a single branch with one admin team (taken from the questionnaire sent to the owner and from the team's visits as customers).
- Azure may not be used for this assignment, which forced the move to Render and MongoDB Atlas (Section 6).
- Hosting must stay within free tiers for the assessment; production costs are given separately in Section 10.
- Business rules such as opening hours, prices and the cancellation window were set to sensible defaults and still need the owner's written confirmation (Section 2.5).

1.8 How to read this document

Section 2 states the requirements. Section 3 states the quality targets. Sections 4 to 7 describe the design and architecture. Section 8 analyses security risk. Section 9 describes the DevOps process. Section 10 gives the running costs and Section 11 the change-management roadmap. Section 12 links the prototype and live system, and the references follow. A glossary is not needed: terms are explained where they first appear.

2. Requirements

This section lists who uses the system, what each person needs to do, the journeys they follow, the work needed to hand the system to the client and the assumptions that remain open. Every item carries one status so that the section does not contradict itself.

2.1 User roles

The roles were derived from the client questionnaire, from the team's visits as customers and from the way the shop works. The owner has been kept informed of progress (most recently on 6 October 2026), but no written sign-off was received before submission. The roles are therefore recorded as **working roles, client sign-off pending**, and the same status is used for the user stories in Section 2.2.

Role	What they can do	How they are identified	Status
Guest	Browse services, barbers and products; must sign in to book or review.	No account	Delivered
Customer	Book, reschedule, cancel, review and rebook; reserve products; manage a profile; receive emails.	Auth0 account, role claim Customer	Delivered
Barber	View own schedule; confirm, complete or mark no-show; add and remove time blocks.	Auth0 account linked to a barber profile, role claim Barber	Delivered
Admin	Manage services, barber profiles and skills, products and stock; view all bookings; manage staff roles; view the dashboard.	Auth0 account, role claim Admin	Delivered

2.2 User stories

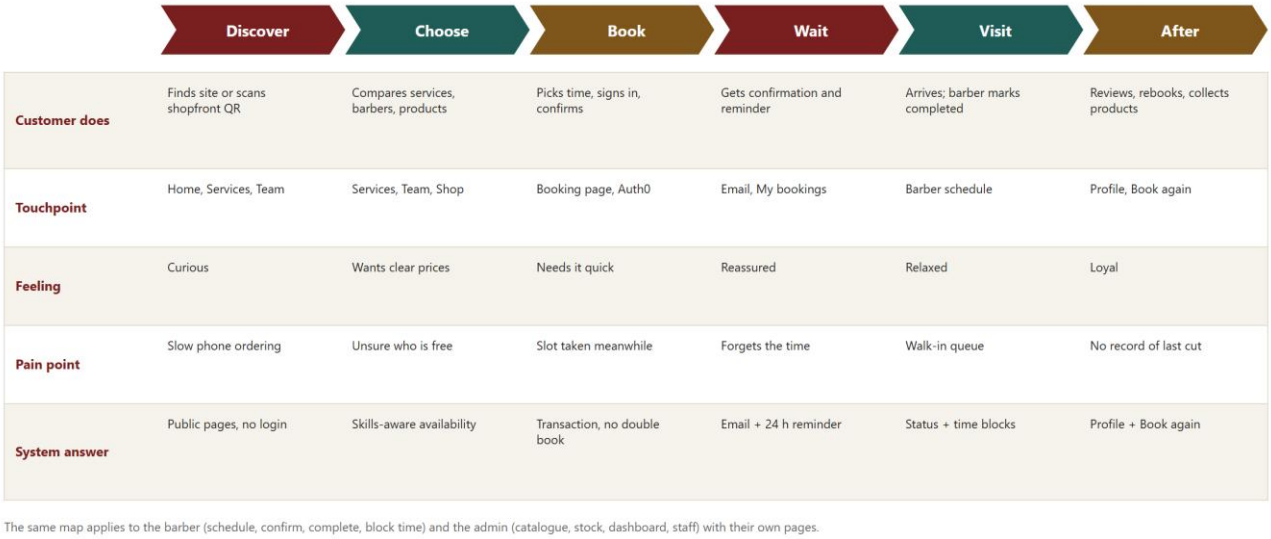
Each story has a priority (MoSCoW: Must, Should, Could), acceptance criteria that can be tested, and a status. “Not built” means the story was written in Task 1 and later moved out of scope.

ID	Role	I want to…	Acceptance criteria	Priority	Status
G1	Guest	view services and prices without an account	Services page lists only active services with price and duration.	Must	Delivered
G2	Guest	view barber profiles	Team page lists active barbers and what each does.	Must	Delivered
G3	Guest	view shop products and prices	Shop page lists active products with price and stock state.	Should	Delivered
G4	Guest	be asked to sign in only when I book	Booking redirects to Auth0 and returns to the booking page.	Must	Delivered
C1	Customer	select several services in one booking	Up to ten services; total duration and price are calculated by the API.	Must	Delivered
C2	Customer	choose a barber or any available barber	Only barbers skilled for every chosen service are offered.	Must	Delivered
C3	Customer	see only free times	Slots exclude bookings, time blocks and working-hours gaps; 15-minute steps.	Must	Delivered
C4	Customer	reschedule or cancel from my account	Allowed until 60 minutes before the start; the slot is released.	Must	Delivered

ID	Role	I want to…	Acceptance criteria	Priority	Status
C5	Customer	limit my open bookings	A fourth upcoming booking is refused with a clear message.	Should	Delivered
C6	Customer	leave a rating after a visit	Review allowed only on a completed booking, once.	Should	Delivered
C7	Customer	receive an email when I book and a reminder	Received, confirmed and completed emails; one reminder about 24 hours before.	Should	Delivered
C8	Customer	reserve products with my booking	Stock is held at booking and returned once on cancellation.	Should	Delivered
C9	Customer	manage my profile and rebook a past visit	Profile saves contact details; Book again pre-fills services and barber.	Could	Delivered
C10	Customer	add products to a cart and check out online	Cart, checkout and card payment.	Could	Not built (out of scope)
B1	Barber	see my schedule for a day or week	Only my own bookings are returned.	Must	Delivered
B2	Barber	block time for breaks and days off	Blocks remove slots from search and are refused if they clash with a booking.	Must	Delivered
B3	Barber	confirm, complete or mark no-show	Only valid transitions are accepted; stale edits are rejected.	Must	Delivered
B4	Barber	see the services and notes for an appointment	Booking detail shows services, products and notes.	Should	Delivered
A1	Admin	add, edit and deactivate services	Past bookings keep their original price snapshot.	Must	Delivered
A2	Admin	manage barber profiles and skills	Profile links to a staff account; skills drive availability.	Must	Delivered
A3	Admin	manage products and stock	Stock never goes below zero; low-stock threshold per product.	Should	Delivered
A4	Admin	see every booking	Admin list returns all barbers.	Must	Delivered
A5	Admin	change staff roles safely	Cannot change own role; audit entry written; barber with upcoming bookings cannot be demoted.	Should	Delivered
A6	Admin	see how the shop is performing	Dashboard shows outcomes, revenue, top services and low stock for a period.	Should	Delivered
A7	Admin	reward repeat customers	Loyalty points and offers.	Could	Not built (out of scope)
G5	Guest	book without creating an account	Guest checkout.	Could	Not built (out of scope)
C11	Customer	receive WhatsApp reminders	WhatsApp message with booking details.	Could	Future
A8	Admin	take payment in the system	Point-of-sale for in-store products.	Could	Future

2.3 User experience journey map

A journey map follows one person through the stages of a goal and records what they do, what they touch, how they feel, what goes wrong and how the system helps. The map below follows a customer from first discovering the shop to returning for another visit.



	Discover	Choose	Book	Wait	Visit	After
Customer does	Finds site or scans shopfront QR	Compares services, barbers, products	Picks time, signs in, confirms	Gets confirmation and reminder	Arrives; barber marks completed	Reviews, rebooks, collects products
Touchpoint	Home, Services, Team	Services, Team, Shop	Booking page, Auth0	Email, My bookings	Barber schedule	Profile, Book again
Feeling	Curious	Wants clear prices	Needs it quick	Reassured	Relaxed	Loyal
Pain point	Slow phone ordering	Unsure who is free	Slot taken meanwhile	Forgets the time	Walk-in queue	No record of last cut
System answer	Public pages, no login	Skills-aware availability	Transaction, no double book	Email + 24 h reminder	Status + time blocks	Profile + Book again

The same map applies to the barber (schedule, confirm, complete, block time) and the admin (catalogue, stock, dashboard, staff) with their own pages.

Figure 1: Customer journey map for Kinsmen Barbershop

Journeys for the other roles

Role	Journey (screens in order)
Guest	Home → Services or Team or Shop → Book a chair → Auth0 sign-in or register → Booking page.
Customer	Home → Booking (services, barber, date, time, products) → Confirm → My bookings (view, reschedule, cancel) → Profile → Book again.
Barber	Sign in → My Schedule (day or week) → booking detail → Confirm, Complete or No-show → Add or remove a time block.
Admin	Sign in → Admin (services, barbers and skills, products and stock) → Dashboard (choose period) → Staff accounts (change a role, view audit).

2.4 Work breakdown structure for handover

A work breakdown structure (WBS) divides the finished product into levels, from the whole product down to deliverables. The original Task 1 table listed project activities with durations and predecessors, which is a schedule, not a WBS. This revision gives the WBS of the system as it is handed to Kinsmen, so each work package is something the shop (or a future developer) can own, run and maintain.

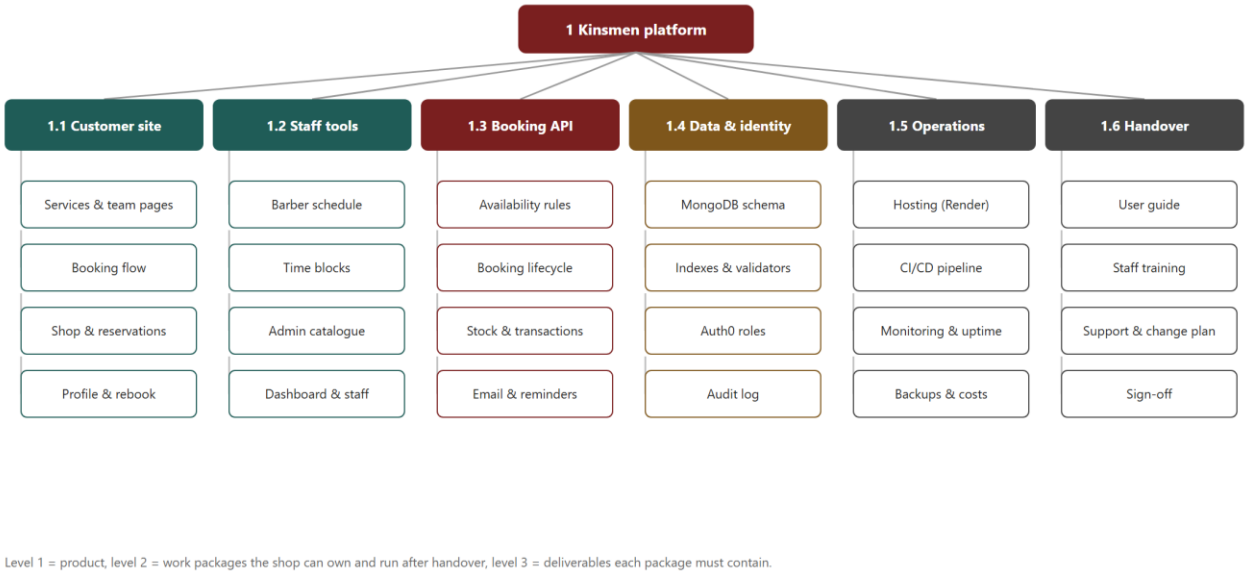


Figure 2: Three-level work breakdown structure of the Kinsmen platform at handover

Code	Work package	Deliverables (level 3)	Owner after handover
1.1	Customer site	Services and team pages; booking flow; shop and reservations; profile and Book again	Developer on retainer
1.2	Staff tools	Barber schedule; time blocks; admin catalogue; dashboard and staff management	Owner and developer
1.3	Booking API	Availability rules; booking lifecycle; stock and transactions; email and reminders	Developer on retainer
1.4	Data and identity	MongoDB schema; indexes and validators; Auth0 roles; audit log	Developer on retainer
1.5	Operations	Render hosting; CI/CD pipeline; monitoring and uptime; backups and costs	Developer and owner
1.6	Handover	User Guide; staff training; support and change plan; sign-off	Owner

The project schedule that produced these deliverables is not repeated here. It is recorded in the meeting minutes and the pull-request history in the repository.

2.5 Assumptions register

The items below were assumed while building the system. Each has an owner and a way to close it, so none is left implied.

Assumption	Value used	Status	How it closes
Opening hours	Mon–Sat 09:00–18:00; Sun 09:00–15:00	Client sign-off pending	Owner confirms in writing; admin edits barber hours.
Service names, prices, durations	Demo values replaced by shop data in the Admin screen	Client sign-off pending	Owner supplies the price list.
Notice and horizon	60-minute booking and cancellation notice; 30-day horizon; max 3 open bookings	Default, adjustable	Owner approves or changes the policy.

Assumption	Value used	Status	How it closes
Product list and stock counts	Products entered by the team	Awaiting shop data	Shop staff send the list and quantities (requested 6 October 2026).
Payment	Paid in store; no online payment	Decided	Revisit in the roadmap (Section 11).
No-show and stock	A no-show does not return reserved stock	Open	Owner decides whether stock returns.

3. Non-functional requirements

Non-functional requirements describe how well the system must work rather than what it does. Each one below has a measure and a way it was or will be verified, so that it can be marked as met or not met.

ID	Quality	Requirement and measure	How it is verified
N1	Performance	Availability search returns in under 2 seconds and pages load in under 3 seconds on a normal mobile connection, with a single-shop data set.	Manual timing on the live site; API tests on realistic data.
N2	Scalability	Web and API run as separate containers so each can scale on its own; the database moves from the free tier to a larger tier without code changes. Target: 100 customers at launch, 10,000 without redesign.	Architecture review (Section 7.3); cost tiers (Section 10).
N3	Availability	Target 99.5% uptime during trading hours (Mon–Sat 09:00–18:00) on a paid always-on plan. On the free assessment tier the first request after 15 minutes idle takes about a minute; this is hidden by a retry handler and wake-up page.	Health endpoints; uptime monitor; manual test after idle.
N4	Maintenance window	Releases are deployed outside trading hours where possible. This protects customers during the day; it does not mean developers only work at night. Development happens at any time on branches and only the release is timed.	Release log; CI deploys only commits that pass checks.
N5	Security	HTTPS everywhere; no passwords stored; role and ownership checked on every API request; secrets only in environment settings; security headers and a Content Security Policy.	Risk register (Section 8); security tests.
N6	Usability	A first-time customer completes a booking in five steps (services, barber, time, products, confirm). Pages work on mobile and desktop.	Walkthrough with team members who were not on the feature.
N7	Reliability	Two requests cannot book the same barber at overlapping times. This is enforced by a multi-document MongoDB transaction that re-checks overlaps against existing bookings and time blocks, plus a version number and an idempotency key (Section 5.1).	Real-MongoDB integration tests for races and rollback.
N8	Maintainability	Layered code, interfaces over data stores, and every change through a reviewed pull request with passing checks.	CI history; pull-request reviews.
N9	Compatibility	Current and previous major versions of Chrome, Edge, Firefox and Safari; no plugin required.	Manual checks in two desktop and two mobile browsers.
N10	Data protection	Only the data needed to run bookings is stored (name, contact, booking history). Times are stored in UTC and shown in Africa/Johannesburg time.	Schema review; POPIA considerations in Section 8.

The original Task 1 report claimed that double booking was prevented “by TimeSlot validation logic”. No such class exists in the final design. Availability is calculated from barber working hours, time blocks and existing bookings, and the transaction in N7 is the real control.

4. Analysis artefacts

This section models the business concepts the system must store and the relationships between them. It explains each class in the final design and records why the model changed from Task 1.

4.1 Domain class diagram

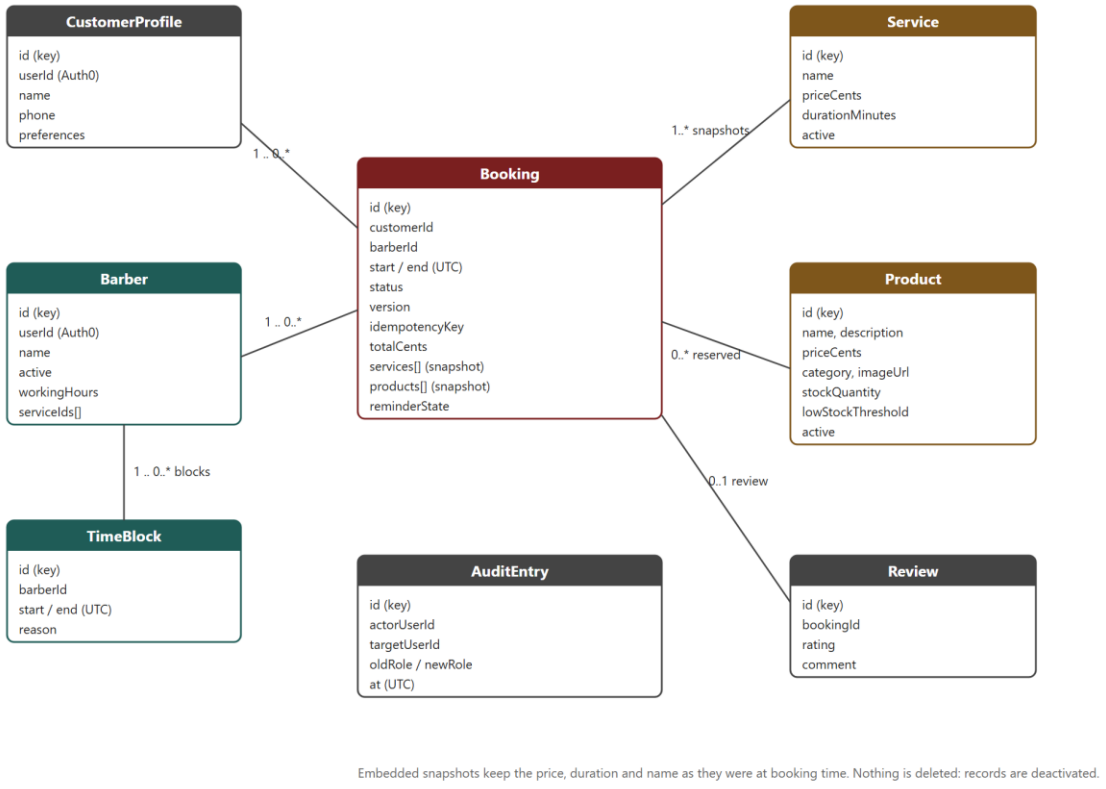


Figure 3: Domain class diagram of the Kinsmen system as built

Classes

Class	Explanation
Booking	The central record. It links a customer and a barber to a start and end time (stored in UTC) and carries the status, a version number that rises with every change, an idempotency key that makes a retried request safe, the total price, the reminder state, and embedded snapshots of the services and products. The snapshots keep the name, price and duration as they were when the booking was made, so later price changes cannot rewrite history or distort the dashboard.
Service	A bookable offering with a name, price in cents, duration in minutes and an active flag. Services are deactivated instead of deleted because bookings refer to them.
Barber	A staff profile linked to one Auth0 account. It holds working hours and the list of services this barber can perform, which the availability search uses so that a customer is never offered a barber who cannot do the chosen service.
TimeBlock	A period in which a barber cannot be booked (a break or day off). It is included in the same overlap calculation as bookings.
Product	A shop item with description, category, image, price, stock quantity, low-stock threshold and active flag. Stock is changed only in a transaction.
CustomerProfile	Contact details and preferences for a signed-in customer. It supports the profile page and Book again.

Class	Explanation
Review	A rating and comment tied to a completed booking.
AuditEntry	A record of each staff-role change: who did it, who it affected, the old and new role, and when.

Relationships

- A **customer** has zero or more bookings; a **barber** has zero or more bookings and zero or more time blocks.
- A **booking** contains one to ten service snapshots and zero or more product snapshots. The snapshots are copies, not links, which is why the relationship has no junction class.
- A **booking** has at most one review, and only once it is completed.
- An **audit entry** refers to account identifiers, not to bookings.

What changed from the Task 1 model

- **Cart, CartItem, Checkout and CheckoutItem were removed.** Products are reserved with a booking and paid for in store, so no cart or checkout exists.
- **BookingService and the TimeSlot class were replaced by embedded snapshots and the TimeBlock class.** A document database stores the booking and its services together, which makes a booking one atomic write.
- **CustomerProfile and AuditEntry were added** for the profile page and the staff-management audit trail.
- Loyalty points remain outside the model (Section 11).

4.2 Design decisions

Decision	Reason
Money stored as integer cents	Avoids rounding errors in totals and revenue.
All times stored in UTC	One unambiguous format; converted to Africa/Johannesburg only for display and emails.
Nothing deleted, only deactivated	Keeps history valid and lets the owner undo a mistake.
Snapshots inside bookings	Protects reports and receipts from later price or name changes.
Version number on every booking	Detects two people editing the same booking and refuses the stale edit.

5. Implementation

This section shows how the system behaves over time. It gives one sequence diagram for each role that uses the system, the booking lifecycle as a state diagram, and the plan used to deploy it.

5.1 Sequence diagrams

A sequence diagram shows who talks to whom, and in what order, for one task. The four diagrams below cover the Guest, Customer, Barber and Admin. In every diagram the web application passes the user's token to the API, and only the API reads or writes the database.

Guest: browse and sign in

A guest can read services, barbers and products without an account. The API returns only active records. The guest meets a sign-in page only when they start to book, and Auth0 returns a signed token that carries their role.

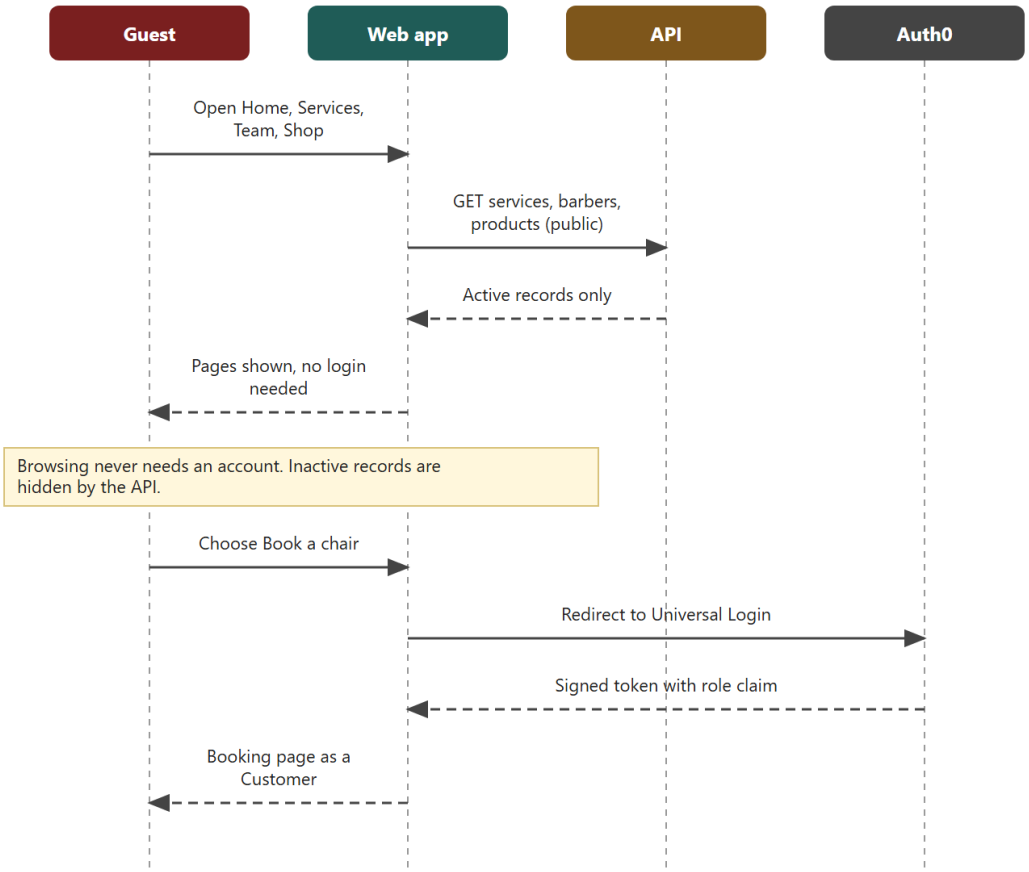


Figure 4: Guest browses and signs in

Customer: create a booking

This is the critical flow. The customer picks services, a barber (or any barber) and a time. The API recalculates the price and duration from the catalogue instead of trusting the browser, then runs one MongoDB transaction that re-checks the barber's overlaps, creates the booking and reserves any chosen products. If another customer took the slot first, the transaction fails and the customer is asked to pick another time. After the commit the API asks Brevo to send the booking email.

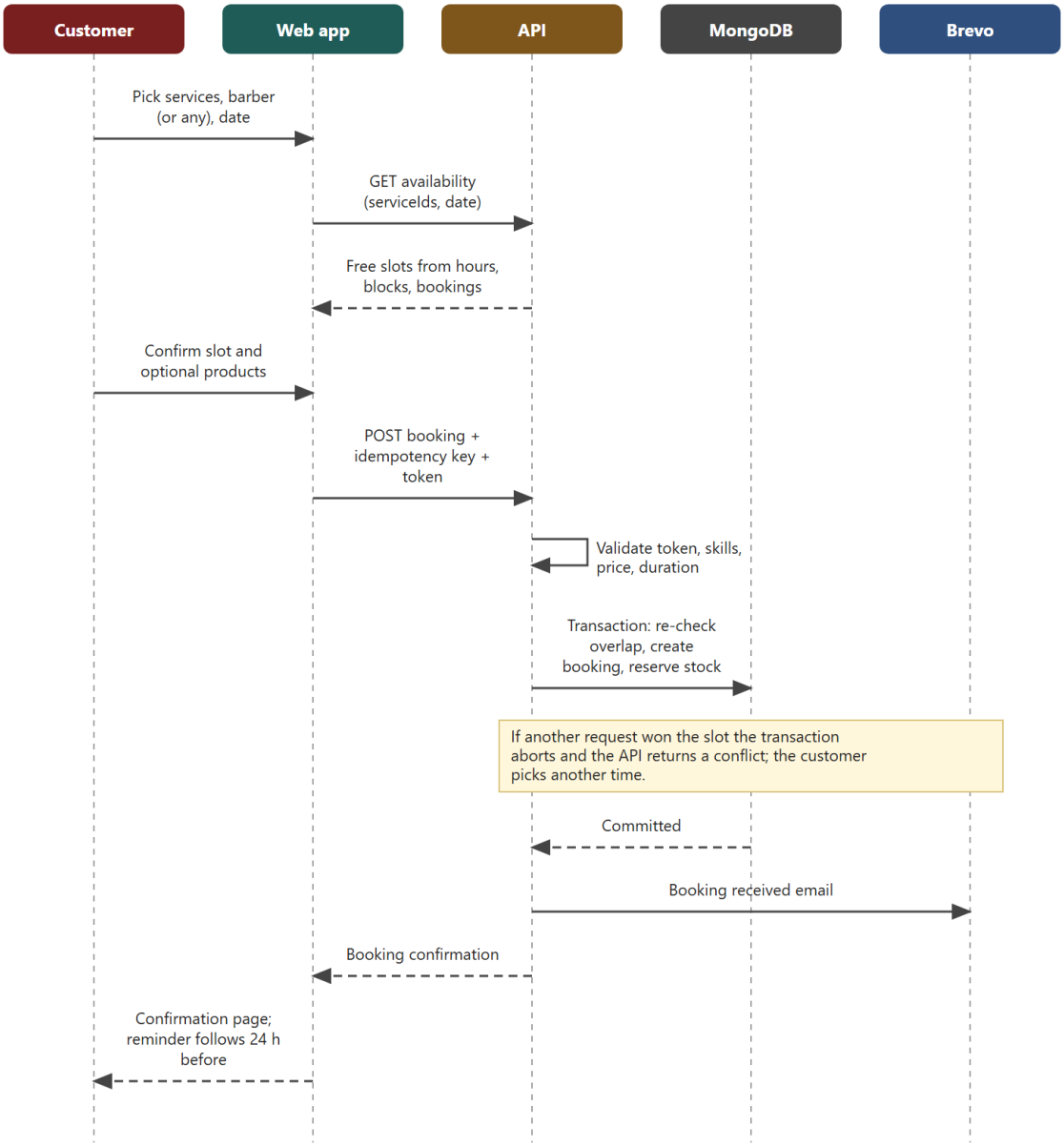


Figure 5: Customer creates a booking

Barber: manage the schedule

A barber sees only their own bookings. Each status change is checked against the allowed transitions and the booking's version number, so two devices cannot overwrite each other. A time block is refused if it overlaps an existing booking.

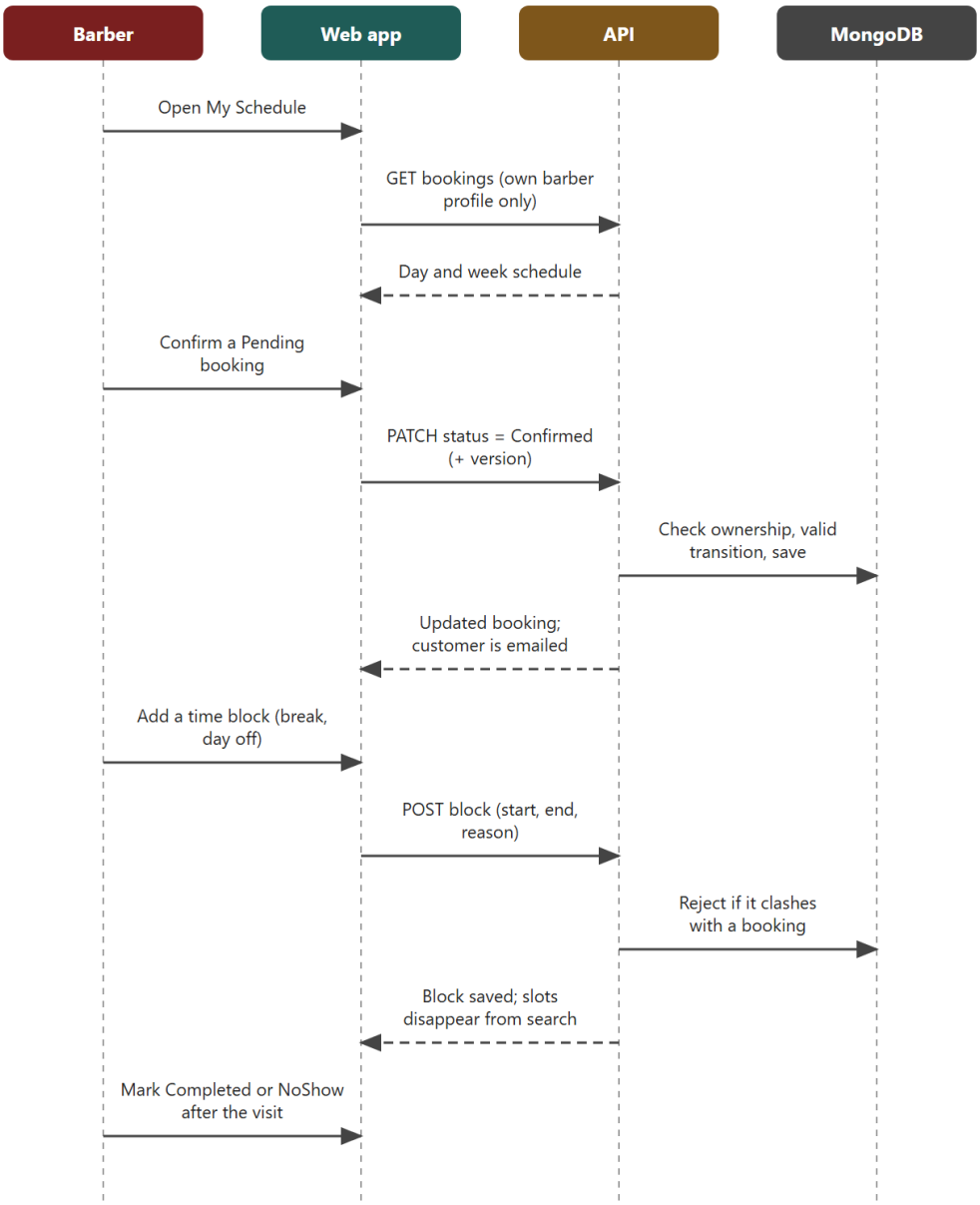


Figure 6: Barber confirms a booking and blocks time

Admin: run the shop

An admin edits the catalogue, adjusts stock, reads the dashboard and manages staff roles. Role changes call the Auth0 Management API with restricted credentials and always write an audit entry.

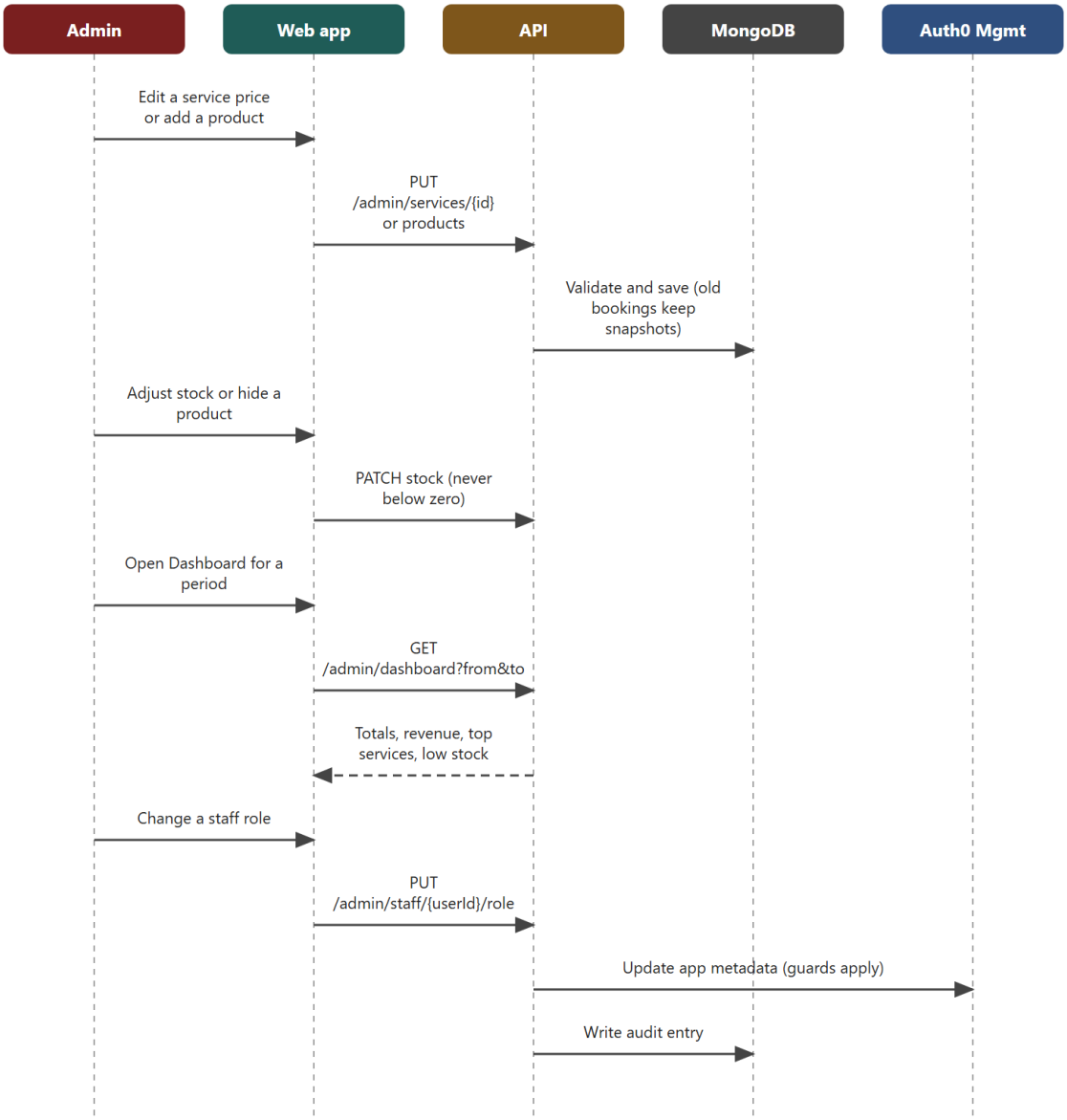


Figure 7: Admin edits the catalogue, reads the dashboard and changes a role

5.2 State diagram: booking status

A booking is always in exactly one state. The diagram shows the states and the events that move a booking between them; the table lists who may trigger each move and the rule that guards it.

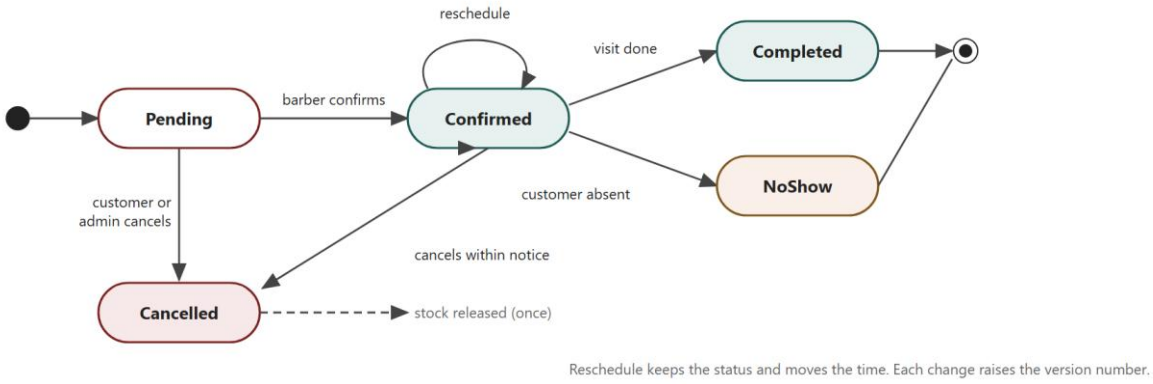


Figure 8: Booking status lifecycle

Transition	Who	Guard	Side effect
Pending → Confirmed	Assigned barber or admin	Booking is Pending; version matches	Confirmation email
Confirmed → Completed	Assigned barber or admin	Start time has passed; version matches	Completed email; review allowed
Confirmed → NoShow	Assigned barber or admin	Start time has passed	None; reserved stock is not returned (open decision)
Pending/Confirmed → Cancelled	Owning customer or admin	At least 60 minutes before start	Slot released; reserved stock returned once
Reschedule (same state)	Owning customer or admin	Notice rule; new slot free; version matches	Version rises; reminder state resets

5.3 Deployment plan

This plan explains where the system runs and the order in which it was deployed. Section 7.3 explains why these services were chosen.

Environments

Environment	Where	Data	Purpose
Local	Developer laptop, dotnet run, Docker MongoDB replica set	Throwaway local database	Building and debugging a feature.
CI	GitHub Actions runner	Throwaway MongoDB replica set created per run	Build, test, container smoke test on every pull request.
Staging	Render, Frankfurt, deployed from the integration branch	MongoDB Atlas, database kinsmen_dev	Team review and demonstrations while features are integrated.
Submission / production	Render, Frankfurt, deployed from main	MongoDB Atlas	The version that is marked and, later, handed to the client.

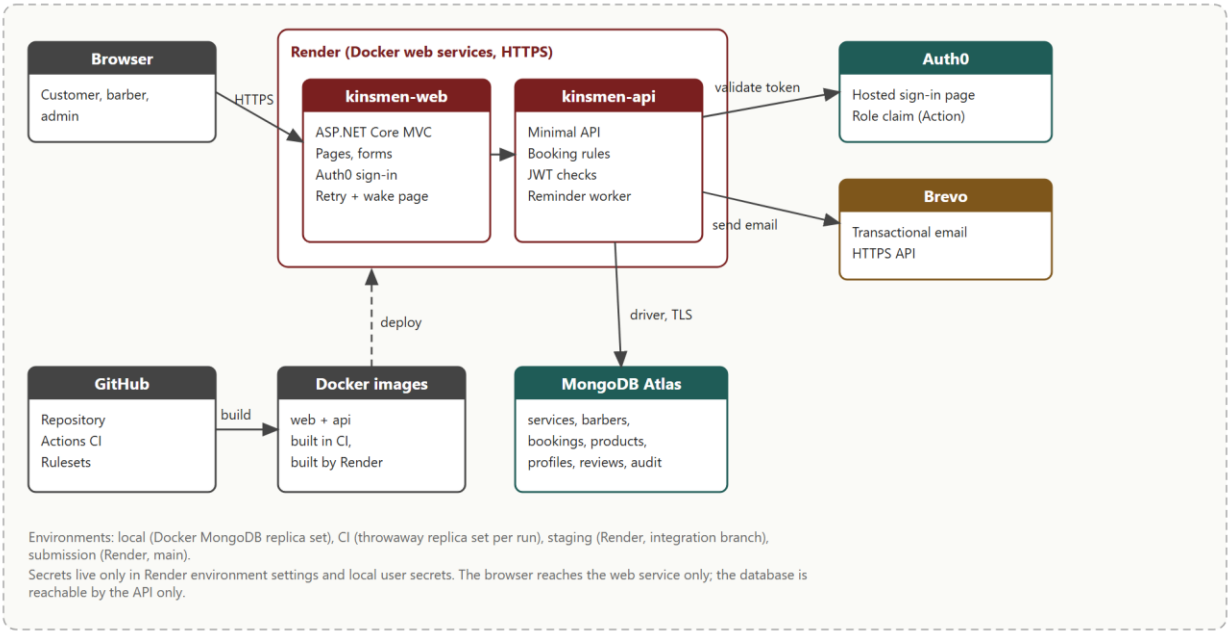
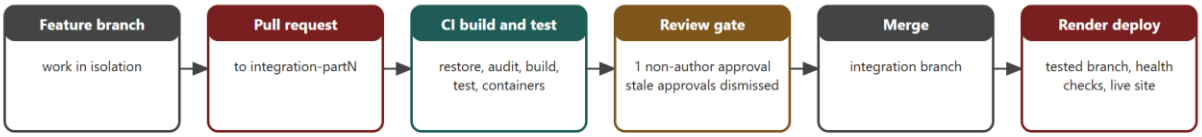


Figure 9: Deployment environment and the services it uses



At the end of the task the integration branch is merged into main (the submission build) and Render is switched to main.

A failed check or a missing approval blocks the merge, so a broken build never reaches the live site.

Figure 10: From feature branch to live deployment

Deployment steps

The steps below were followed in this order. The first four create the services the application depends on, so the application can start and pass its health check.

1. **Create the database.** In MongoDB Atlas create a cluster, a database user restricted to the Kinsmen database and a network rule that lets Render connect.
2. **Initialise the schema.** Run the database initialisation script once. It creates the collections, JSON-schema validators and indexes.
3. **Configure identity.** In Auth0 create the web application and the API, add the post-login Action that writes the role and email into tokens, and set the callback and logout URLs.
4. **Configure email (optional).** Verify a sender address in Brevo and create an API key. Without it the system sends no email but runs normally.
5. **Create the Render services.** Create the blueprint from render.yaml. It defines kinsmen-web and kinsmen-api, the Frankfurt region, the branch to deploy and the shared environment group that holds the proxy key.
6. **Enter the secrets.** Type the MongoDB connection string, Auth0 settings and Brevo key into Render's environment settings. Secrets are never committed to Git.
7. **Deploy.** Render builds both Docker images and starts them only after the GitHub checks pass.
8. **Verify.** Open /health/live and /health/ready on the API, then run the smoke test: sign up, book, reschedule, cancel, confirm as a barber, edit as an admin.
9. **Load real data.** Replace demo services, link each real barber to their account and enter products and stock.
10. **Keep the demo awake.** Use an uptime monitor on both services around demonstrations and marking.
11. **Switch to main.** At submission change the Render branch from the integration branch to main.

Rollback. If a deployment misbehaves, Render can redeploy the previous successful build, and the Git history lets the team revert the merge and redeploy. Schema changes are made additively where possible, so an older build can still run against a newer database.

6. Data schema

This section explains why MongoDB was chosen, what it stores and how the data is kept correct.

6.1 Choice of technology

Task 1 proposed Azure SQL with Entity Framework Core. The group was not allowed to use Azure, so the database had to be chosen again. Five options were compared against six criteria. Each criterion was weighted by how much it mattered to the project (3 = essential, 1 = nice to have), and each option was scored from 1 (poor) to 5 (excellent) by the team after reading the documentation listed in the references.

Option	Stores a booking with its service snapshots as one unit (w3)	Multi-document transactions for booking and stock (w3)	Usable free tier for the assessment (w3)	Official, current .NET driver (w2)	Clear path to scale (w2)	Team could learn it in the time available (w1)	Weighted total
MongoDB Atlas	5	5	5	5	4	4	67
Azure SQL + EF Core	3	5	1	5	4	5	50
PostgreSQL (hosted) + EF Core	3	5	3	5	4	4	55
Firestore	4	3	4	2	5	3	50
SQLite file	3	4	5	5	1	5	53

The maximum possible total is 70. MongoDB Atlas scored highest. Azure SQL scored one for the free tier because Azure could not be used at all. The scores are the team's judgement, not a measurement, so the reasons are given below.

- **Booking shape.** A booking carries the services and products as they were priced on the day. Storing them inside the booking is natural in a document database and avoids joins when a customer opens their history.
- **Transactions.** MongoDB supports multi-document ACID transactions on replica sets, including Atlas free clusters (MongoDB, n.d.-a). The booking rule needs this: check overlaps, create the booking and reserve stock must succeed or fail together.
- **Free tier.** Atlas M0 is free forever with 512 MB of storage and shared resources (MongoDB, n.d.-b), which is enough for the assessment (see the capacity estimate below).
- **Tooling.** MongoDB publishes and maintains the C# driver used by ASP.NET Core (MongoDB, n.d.-c). Hosted PostgreSQL was a close second but its free tiers have time limits or very small storage, and moving to it would have meant rewriting the data layer a second time.
- **Why not the others.** Azure SQL was not permitted. Firestore has a weaker .NET story and its transaction limits make a booking-plus-stock rule awkward. SQLite cannot be shared by two Render services and does not survive a redeploy.

Scalability and capacity

A busy single shop may take about 40 bookings a day, which is roughly 14,400 a year. A booking document with snapshots is about 2 KB, so a year of bookings is about 30 MB, well under the 512 MB free allowance. The path to a larger audience has three steps that need no code change: move to the Flex tier (5 GB, billed by usage), then to a dedicated M10 cluster (from about US$57 a month) that adds backups and more memory, and, only if a very large audience ever required it, shard the bookings collection by barber. The indexes below keep availability searches fast as the collection grows.

6.2 Collections and indexes

Collection	Key fields	Indexes and rules
services	name, priceCents, durationMinutes, active	Validator: required fields, price and duration above zero.
barbers	userId, name, active, workingHours, serviceIds	Unique index on userId: one profile per staff account.
bookings	customerId, barberId, startUtc, endUtc, status, version, idempotencyKey, totalCents, services, products, reminder state	Index (barberId, startUtc, endUtc) for overlap checks; index (customerId, startUtc) for My bookings; status limited to the allowed vocabulary.
timeBlocks	barberId, startUtc, endUtc, reason	Index (barberId, startUtc, endUtc).
products	name, description, category, imageUrl, priceCents, stockQuantity, lowStockThreshold, active	Stock is an integer that cannot go below zero.
customerProfiles	userId, name, phone, preferences	One profile per account.
reviews	bookingId, barberId, rating, comment, createdUtc	Unique index on bookingId (one review per booking); index (barberId, createdUtc desc).
auditLog	actorUserId, targetUserId, oldRole, newRole, at	Append-only.

6.3 Data integrity

MongoDB has no foreign keys, so integrity is enforced in three layers. The API validates every reference before it writes. JSON-schema validators in the database reject documents with missing fields or wrong types. Transactions make multi-document changes atomic. Together these give the same protection that foreign keys and constraints gave in the original relational design.

7. Architecture artefacts

This section names the patterns used in the code, shows how they work together, and justifies the cloud architecture.

7.1 Design patterns

A design pattern is a proven way of solving a recurring problem. The patterns below are the ones that are actually used in the code. Patterns proposed in Task 1 that were not used (unit of work, factory) have been removed.

Pattern	Where it is used	Why it helps
Repository (store interface)	IBookingStore with MongoBookingStore and a test store; the services depend only on the interface.	The booking rules run against an in-memory store in unit tests and against MongoDB in integration tests.
Transaction script / service layer	BookingService, CatalogueService, ReportService hold the business rules.	Rules sit in one place, not in controllers or views.
Strategy	The email sender is chosen at start-up: Brevo when configured, a no-op sender otherwise.	Development and tests run without sending real email.
Background worker	ReminderWorker sweeps every 10 minutes and claims each reminder atomically before sending.	A restart cannot send the same reminder twice.
Dependency injection	ASP.NET Core registers stores, clocks and senders.	Each piece can be replaced in tests.
Decorator (delegating handler)	ColdStartRetryHandler wraps the web app's API client.	Retries read requests while the sleeping API wakes, without changing the client code.

Notifications. Email is the only notification channel built. It was chosen because it works over HTTPS from the free hosting tier and needs no extra approval. WhatsApp was discussed as an option, but it is not part of the system and has not been agreed with the client. If the client wants it, it would be added behind the same email-sender interface.

7.2 Architecture patterns

The patterns above fit inside three architectural choices: model-view-controller (MVC) in the web app, a layered API behind it, and a client-server split between them. The trace below follows one booking request through every layer to show how they connect.

Step	Layer	What happens
1	View (Razor + JavaScript)	The customer picks services and a time; the page posts the choice to the web controller.
2	Controller (MVC)	BookingController checks the form, then calls the API client with the user's access token.
3	API client (decorator)	The retry handler waits briefly if the API is asleep, then sends the request.
4	Endpoint	The minimal API validates the token and role and passes the request to the service.
5	Service layer	BookingService recalculates price and duration, checks skills and notice rules.

Step	Layer	What happens
6	Repository	MongoBookingStore runs the transaction that re-checks overlaps, writes the booking and reserves stock.
7	Outbound	The email sender asks Brevo to send the confirmation; the response travels back up to the view.

The layers keep each concern in one place. If the owner later wants a different payment or email provider, only the sender or a new service changes; the pages and the booking rules do not.

7.3 Cloud architecture

The system was built on managed cloud services so that the shop has no servers to look after. Figure 9 in Section 5.3 shows the finished architecture. The table gives the reason for each choice and the alternative that was considered.

Component	Chosen	Alternatives considered	Reason
Hosting	Render Docker web services, Frankfurt region	Azure App Service (not permitted); other container hosts	Runs the existing Docker images, deploys from GitHub after checks pass, has a free tier and keeps web and API separate.
Database	MongoDB Atlas	See Section 6.1	Transactions, free tier, official .NET driver.
Identity	Auth0 Universal Login	ASP.NET Identity	Passwords and recovery stay with a specialist; free for up to 25,000 monthly active users (Auth0, n.d.).
Email	Brevo HTTPS API	SMTP	Free hosting tiers do not allow outbound SMTP; the free plan covers 300 emails a day.
Region	Frankfurt	Oregon, Singapore, Ohio	Nearest Render region to South Africa. The Atlas cluster should be created in a nearby European region so that API-to-database calls stay fast.
Resilience	Health endpoints; retry handler; wake page	None	Hides the free tier's sleep from users; a paid plan removes it.

The architecture is single-region and has one instance of each service. That is appropriate for one shop. If availability became critical, the next steps would be an always-on paid plan, a second web instance and Atlas backups (Section 11).

8. Security

This section lists what could go wrong, rates each threat formally and shows the control that reduces it. A risk is rated by multiplying its likelihood by its impact, each scored from 1 to 5, before and after controls are applied. The result is the residual risk the team accepted.

8.1 Rating scales

Score	Likelihood (in a year of operation)	Impact
1	Rare: not expected	Negligible: no customer notices
2	Unlikely: could happen once	Minor: small inconvenience
3	Possible: may happen	Moderate: bookings or stock wrong, fixable
4	Likely: probably happens	Major: personal data exposed or shop cannot trade for a day
5	Almost certain: will happen	Severe: widespread data loss or full system compromise

Rating bands: 1–4 Low, 5–9 Medium, 10–14 High, 15–25 Critical.

8.2 Risk register

ID	Threat	L	I	Score	Control	Resid. L×I	Residual
R1	A customer reads or changes another customer's booking	2	4	8 Medium	JWT validation and subject-based ownership checks in the API on every request	1×4	4 Low
R2	A user gives themselves the Admin or Barber role	2	5	10 High	Roles live in Auth0 app metadata and signed tokens; staff changes use a restricted machine-to-machine app; no self role change; audit log	1×5	5 Medium
R3	Database credentials or API keys leak through the public repository	3	5	15 Critical	Secrets only in Render settings and local user secrets; reviewed pull requests; example config contains names only	1×5	5 Medium
R4	Two customers book the same barber and time at once	4	3	12 High	Multi-document transaction touching the barber, version numbers, idempotency keys	1×3	3 Low
R5	Script or content injection through notes, names or product image links	3	3	9 Medium	Razor output encoding, server-side validation, Content Security Policy, HTTPS-only image URLs	1×3	3 Low
R6	Free-tier sleep or outage during a demonstration or busy day	5	3	15 Critical	Retry handler, wake-up page, health endpoints, keep-awake monitor; paid always-on plan recommended for go-live	3×2	6 Medium
R7	Email provider fails or the daily limit is reached	3	2	6 Medium	No-op sender when unconfigured, once-only reminder claim, provider dashboard monitoring	2×2	4 Low
R8	Shop stock is oversold or goes negative	3	3	9 Medium	Stock decremented inside the booking transaction; restocked once on cancellation; floor of zero	1×3	3 Low
R9	Personal data is exposed or kept longer than needed (POPIA)	2	4	8 Medium	Data minimisation, no passwords stored, HTTPS everywhere, least-privilege database user	1×4	4 Low
R10	A provider changes price or terms	3	3	9 Medium	Standard Docker, ASP.NET Core and MongoDB keep the system portable; costs reviewed each term	2×3	6 Medium

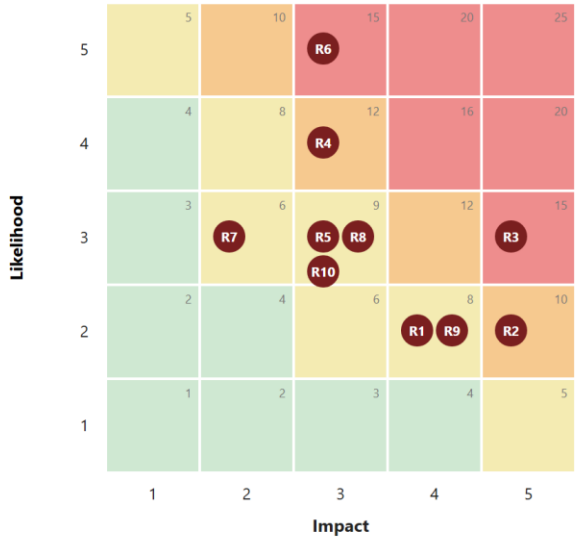


Figure 11: Inherent risk before controls (numbers refer to the register)

After controls no risk is above Medium. The highest residual risk is R6, the free-tier sleep, which is a hosting limit rather than a defect and is removed by moving to a paid plan (Section 10). R2 keeps a residual score of 5 because the impact of a stolen admin role is severe even though it is now very unlikely.

8.3 Controls in detail

Area	Control
Authentication	Auth0 Universal Login handles sign-up, sign-in, verification and password recovery. The system stores no passwords.
Authorisation	The API checks the signed token (issuer, audience, signature, lifetime) and the role claim on every protected request. Ownership checks use the token's subject, never an identifier from the browser.
Transport	HTTPS on both services; secure cookies; forwarded headers handled behind Render's proxy; a shared proxy key between web and API.
Input and output	Server-side validation of every field; Razor encodes output; anti-forgery tokens on form posts; the Content Security Policy blocks inline and foreign scripts and allows images only from the site, data URIs and HTTPS.
Browser hardening	frame-ancestors 'none' stops clickjacking; X-Content-Type-Options: nosniff; limited referrer and permissions policies.
Data	Least-privilege Atlas user; only the API can reach the database; times in UTC; nothing is deleted, so history is intact.
Secrets	Held in Render environment settings and local user secrets; the repository contains names and safe examples only.
Staff management	Restricted machine-to-machine credentials; no self role change; a barber with upcoming bookings cannot be demoted; every change is audited.
Supply chain	Locked package restore and a vulnerable-package check on every build.

8.4 Privacy (POPIA)

The Protection of Personal Information Act requires that personal data is collected for a clear purpose and kept safe. The system collects a name, email and optional phone number to run bookings, never collects payment details, and stores no passwords. Before the client takes the system live the owner should name an information officer, add a short privacy notice to the site and agree how long booking history is kept.

8.5 Future security options

Two options from Task 1 were not built and remain future considerations: a QR code at the shop front that opens the booking page (the page is public, so this carries no extra risk) and passkey (WebAuthn) sign-in, which Auth0 can provide without changes to the booking code.

9. DevOps

DevOps is the set of habits and tools that let a team change software quickly without breaking it. This section describes how code moves from a developer's laptop to the live site and how the team keeps it working.

9.1 Branching and review

The Task 1 plan used one protected main branch. As the team built features in parallel, a three-level flow replaced it:

1. **Feature branches** hold one person's work, for example feature/zario-product-shop or feature/greg-customer-profile.
2. **integration-partN** (integration-part2, then integration-part3) is the shared branch where finished features are combined and checked together. All feature pull requests target it.
3. **main** receives the integration branch only at the end, once everything has been verified. It is the submission build.

A repository ruleset protects the integration and main branches. A change must arrive by pull request, the required checks must pass, at least one person other than the author must approve it, and an approval is dismissed if the base branch changes afterwards. The rule worked as intended: a reviewer found three defects in the shop pull request (unvalidated image links, a stock edit that could overwrite a reservation, and inaccurate copy), the author fixed them, and the pull request was approved only after the changes.

9.2 Continuous integration

The Build and test workflow runs on every pull request. It has two jobs.

Job / step	What it does
Start MongoDB replica set	Creates a throwaway database that supports transactions, so the integration tests run against the real engine.
Restore (locked)	Installs packages exactly as recorded in the lock files, so the build is repeatable.
Vulnerable-package check	Fails the build if a dependency has a known advisory.
Build (Release)	Compiles the solution.
Test	Runs the whole xUnit suite and uploads the results. A failure blocks the merge.
Container images job	Builds both Docker images and smoke-tests them with production-like settings, so a broken Dockerfile is caught before deployment.

9.3 Automated testing

The suite has more than 300 tests. Unit tests check the business rules without a database: price and duration, notice limits, status transitions, ownership, stock reservation and the dashboard maths. Integration tests run against a real MongoDB replica set and cover concurrent bookings, rollback and stock that never goes negative. Web tests check controller behaviour, role-based pages and security headers. A small group of tests that need the internet (the live Auth0 round trip) run only when asked and are skipped in CI.

9.4 Continuous deployment

Render builds and deploys both services from the chosen branch, but only for commits whose GitHub checks passed (autoDeployTrigger: checksPass). The branch, region and plan are written in render.yaml, so the

environment can be recreated from the repository. During Task 3 Render tracked integration-part3; for submission it was switched to main.

9.5 Configuration and secrets

Item	How it is handled
Database connection string, Auth0 and Brevo keys	Entered in Render's environment settings and in local user secrets; never in Git.
Shared proxy key	One environment group (kinsmen-internal) is attached to both services, so web and API agree on it.
Non-secret settings	Written in render.yaml and appsettings and reviewed in pull requests.
Example configuration	Committed with names only, so a new developer knows what to set.

9.6 Monitoring, release and rollback

- **Health:** /health/live shows the process is up; /health/ready also pings MongoDB. Render uses them to detect a failed start.
- **Uptime:** an external monitor pings both services around demonstrations so the free tier does not sleep.
- **Release:** merge to the integration branch, let the checks pass, let Render deploy, run the smoke test (sign up, book, reschedule, cancel, barber action, admin action).
- **Rollback:** redeploy the previous successful build in Render, or revert the merge in Git. Schema changes are made additively where possible, so an older build can still run against the current database.
- **Definition of done:** the change is reviewed, tests are added or updated, checks are green and any documentation affected is updated.

10. Running costs

This section estimates what the system costs to run. It gives the cost during the assessment, the cost of a sensible first year for the shop, and the cost as the audience grows. All amounts are estimates and exclude VAT, bank charges and any custom development.

10.1 Exchange rate and sources

Providers price in US dollars. The planning rate used here is **R16.50 per US dollar**. The latest dated mid-market figures found were R16.49 to R16.51 on 22 and 31 July 2026; a rate dated November 2026 could not be confirmed, so every amount must be recalculated at the rate on the day of purchase. The Task 1 report used R18.5, which is no longer realistic. A 10% move in the rate changes every total in this section by about 10%.

Prices were read from the providers' pages on 4 November 2026: MongoDB Atlas (Flex up to US$30 a month with a US$8 floor; dedicated M10 from US$56.94 a month), Auth0 (free up to 25,000 monthly active users; the Essentials consumer plan from US$35 a month) and Render (750 free instance hours a month, sleep after 15 minutes of inactivity, about one minute to wake). Render's paid web service prices (about US$7 for Starter and US$25 for Standard per service per month) and Brevo's (free 300 emails a day; Starter about US$9 a month for 5,000 emails) came from secondary summaries and must be checked on each provider's pricing page before anything is purchased.

10.2 Monthly cost scenarios

Cost line	Assessment	Go-live (about 100 customers)	Growth (about 1,000 customers)	Scale (about 10,000 customers)
Render web + API	R0	R231	R825	R1,238
MongoDB Atlas	R0	R132	R495	R940
Auth0	R0	R0	R0	R0
Brevo	R0	R0	R148	R297
Domain	R0	R13	R13	R13
Monthly total	**R0**	**R376**	**R1,481**	**R2,487**
Annual total	**R0**	**R4,506**	**R17,772**	**R29,838**

Assumptions behind each column.

- **Assessment:** every service on its free tier. The only cost is the team's time.
- **Go-live:** both Render services on the Starter plan so they never sleep, Atlas Flex at its US$8 floor, Auth0 and Brevo on free plans, and a .co.za domain estimated at R150 a year (to be confirmed with a registrar).
- **Growth:** Standard instances for headroom, Atlas Flex near its US$30 ceiling and Brevo Starter because reminders can pass 300 emails a day.
- **Scale:** an extra web instance, a dedicated Atlas M10 (adds backups and dedicated memory) and Brevo Standard. Auth0 stays free until 25,000 monthly active users, so identity costs nothing at this size.

10.3 What the numbers mean for the shop

Going live costs about R376 a month, roughly the price of a few haircuts, and it is the point at which the system stops sleeping. The cost grows slowly because the heavy components (identity and email) are free for a long

time, and because a single barbershop needs very little storage (about 30 MB of bookings a year, Section 6.1). A growth plan can be funded by a small booking fee or by fewer missed appointments: if the system prevents even a handful of no-shows a month, it has paid for itself.

10.4 Costs not included

- Payment processing fees, if online payment is added.
- A paid email domain or branded mailbox.
- Developer time for changes after handover.
- VAT and bank charges.

11. Change management

Change management is how the shop moves from taking bookings by phone to running them through the system. This section sets out the roadmap, how staff and customers will be prepared, and how success will be measured.

11.1 Roadmap

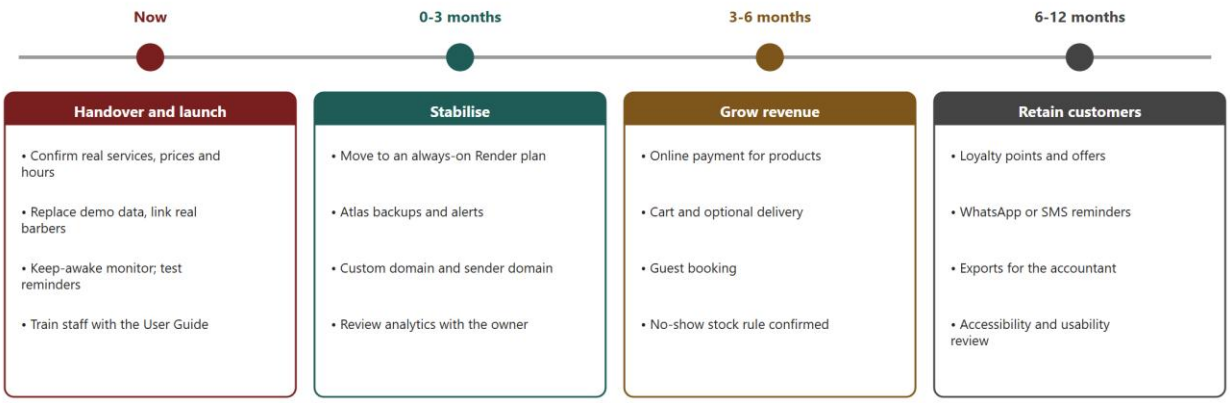


Figure 12: Roadmap from handover to the first year

Phase	Objective	Activities	Owner	Done when
Now: handover and launch	Run the shop on real data	Confirm services, prices, hours and cancellation rules in writing; replace demo data; link each barber's account; enter products and stock; train staff; test reminders	Owner and Kaehil	First real booking made online
0–3 months: stabilise	Make it dependable	Always-on Render plan; Atlas backups and alerts; custom domain and sender domain; weekly review of the dashboard with the owner	Developer	No sleep; one month of uptime above 99.5%
3–6 months: grow revenue	Sell more	Online payment for products; cart and delivery option; guest booking; decide whether a no-show returns stock	Owner and developer	Payment flow passes a security review
6–12 months: retain customers	Bring people back	Loyalty points and offers; WhatsApp or SMS reminders if approved; exports for the accountant; accessibility and usability review	Owner and developer	Repeat bookings rise

11.2 Preparing staff

Who	Session	Length	Covers
Barbers	Schedule and time blocks	30 minutes	Open My Schedule, confirm and complete, block a break, what a no-show does.
Admin / owner	Catalogue and stock	45 minutes	Edit a price, add a product, adjust stock, hide an item, read the low-stock list.
Admin / owner	Dashboard and staff	30 minutes	Choose a period, read totals and top services, change a role and read the audit log.

Who	Session	Length	Covers
Everyone	Support and escalation	15 minutes	What to do if the site is slow, who to call, where the User Guide is.

11.3 Preparing customers

- Put the booking address on the shop door, receipts and social media, and add a QR code at the counter.
- Staff mention online booking whenever a customer books by phone, and book on the customer's phone if they ask.
- The first booking email explains how to reschedule or cancel; the reminder reduces missed appointments.

11.4 Measuring adoption

Measure	Where it comes from	Suggested target
Share of bookings made online	Dashboard totals compared with the shop diary	50% after three months
No-shows per month	Dashboard outcomes	Falling month on month
Cancellations with notice	Booking status history	Rising share of all cancellations
Barber use of the schedule	Barber status updates	Every completed visit marked
Product reservation rate	Reserved products on bookings	Reported monthly

11.5 Adoption risks

Risk	Response
Staff keep using the paper diary	Run both for two weeks, then retire the diary; the owner reviews the dashboard weekly.
Customers are used to phoning	Staff offer to book on the customer's phone; keep a phone number visible.
Free-tier sleep gives a bad first impression	Move to an always-on plan before launch.
Prices or hours change	The admin screen edits them without a developer; past bookings keep their snapshot.

12. Prototype and live system

The clickable prototype from Task 1 and the live system are available at the addresses below. The prototype shows the original design intent, including features that were later moved out of scope (the cart and loyalty); the live system is the final build.

Item	Address
Task 1 prototype (GitHub Pages)	https://kaehil-i.github.io/kinsmen-barbershop/kinsmen_prototype.html
Live system	https://kinsmen-web.onrender.com
API health	https://kinsmen-api.onrender.com/health/ready
Source code (submission)	https://github.com/EMKNDN/emkndn-insy7315-2026-insy7315-project-kaehil-i
Demo video	https://youtu.be/lZX7ETwlSG4

The live system runs on free hosting until go-live, so the first page load after a quiet period can take up to a minute.

References

Auth0 (n.d.) Auth0 pricing. Available at: https://auth0.com/pricing (Accessed: 4 November 2026).

Brevo (n.d.) Brevo pricing plans. Available at: https://www.brevo.com/pricing/ (plan details checked through secondary summaries, 4 November 2026).

Microsoft Learn (n.d.) Entity Framework Core documentation. Available at: https://learn.microsoft.com/ef/core/

Microsoft Learn (n.d.) ASP.NET Core documentation: authentication, authorization, minimal APIs, hosted services. Available at: https://learn.microsoft.com/aspnet/core/

MongoDB (n.d.-a) Transactions. Available at: https://www.mongodb.com/docs/manual/core/transactions/ (Accessed: 4 November 2026).

MongoDB (n.d.-b) MongoDB Atlas pricing. Available at: https://www.mongodb.com/pricing (Accessed: 4 November 2026).

MongoDB (n.d.-c) MongoDB .NET/C# Driver. Available at: https://www.mongodb.com/docs/drivers/csharp/current/ (Accessed: 4 November 2026).

Render (n.d.) Deploy for free. Available at: https://render.com/docs/free (Accessed: 4 November 2026).

Render (n.d.) Render pricing. Available at: https://render.com/pricing (paid instance prices checked through secondary summaries, 4 November 2026).

Republic of South Africa (2013) Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer.

GitHub (n.d.) GitHub Docs: Actions and repository rulesets. Available at: https://docs.github.com/

The earlier Task 1 sources on QR codes, passkeys and design patterns remain valid for the future options noted in Section 8.5 and are listed in the original Task 1 report.